

VERIQTA

YAML

for DevOps

HANDBOOK



KUBERNETES



CI/CD PIPELINES



DOCKER
COMPOSE



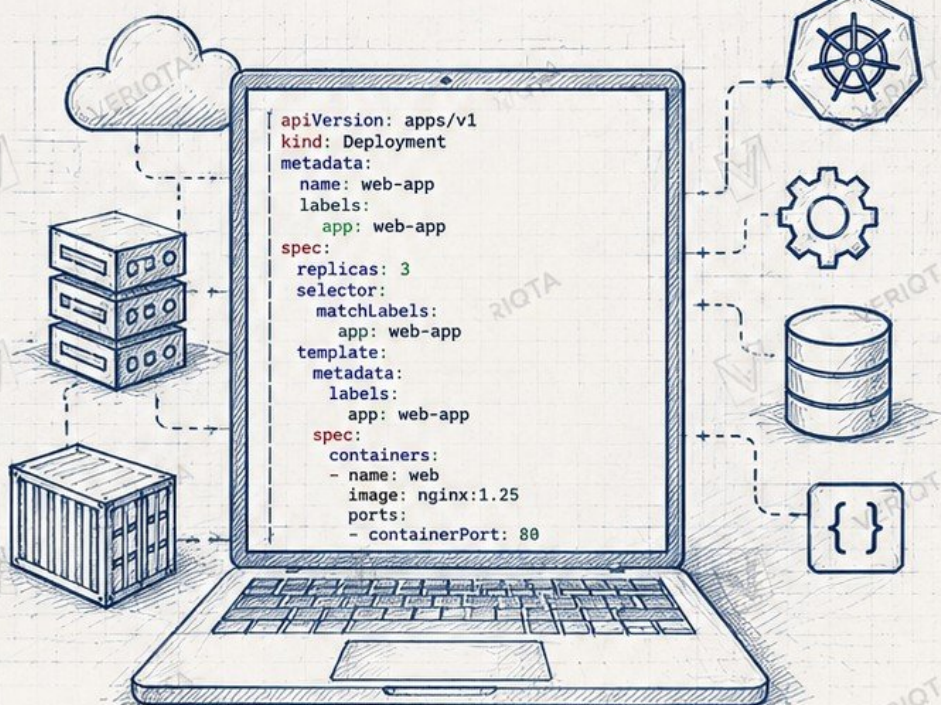
ANSIBLE



GITHUB
ACTIONS



INFRASTRUCTURE
AS CODE



UNDERSTAND
YAML
FUNDAMENTALS



WRITE CLEAN
CONFIGURATIONS



DEBUG AND
TROUBLESHOOT
EASILY



FOLLOW PRODUCTION
BEST PRACTICES



DEPLOY WITH
CONFIDENCE

VERIQTA



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

1. YAML IN MODERN DEVOPS

WHAT YAML IS

YAML (YAML Ain't Markup Language) is a human-readable data serialization format used to store and exchange structured information.



YAML IS:

- ✓ Readable
- ✓ Structured
- ✓ Consistent
- ✓ Easy to learn

YAML IS NOT:

- ✗ A programming language
- ✗ Not executable
- ✗ Not for logic or algorithms
- ✗ Not case-sensitive

★ WHY DEVOPS ENGINEERS USE YAML

- Simple and readable by humans
- Widely supported across DevOps tools
- Ideal for configuration-as-code
- Easy to version control in Git
- Enables automation and reproducibility
- Encourages consistency across environments

</> YAML AS CONFIGURATION, NOT PROGRAMMING

YAML IS FOR CONFIGURATION

- ✓ Defines desired state
- ✓ Describes what you want
- ✓ Tool interprets the YAML
- ✓ No loops, conditions (unless in specific tools)

NOT FOR PROGRAMMING

- ✗ No complex logic
- ✗ No calculations
- ✗ No dynamic execution
- ✗ No flow control (outside tool features)

🌐 WHERE YAML APPEARS IN PRODUCTION



KUBERNETES

Defines cluster resources like Deployments, Services, ConfigMaps, Secrets, Ingress, and more.



CI/CD PIPELINES

Defines pipeline stages, jobs, steps, variables, artifacts, environments, and deployment workflows.



DOCKER COMPOSE

Defines multi-container applications, services, networks, volumes, and environment configuration.



ANSIBLE

Defines playbooks, tasks, variables, handlers, roles, and automation workflows.



GITHUB ACTIONS

Defines workflows, jobs, steps, actions, secrets, runners, and CI/CD automation.



INFRASTRUCTURE & AUTOMATION TOOLS

Used in Terraform, Helm, Argo CD, SaltStack, Prometheus, Vault, and many other tools.

🔍 HOW TO APPROACH UNFAMILIAR YAML FILES

- 1 Identify the purpose → What tool or system uses this YAML?
- 2 Find the root keys → Look for top-level keys like apiVersion, kind, name, version, services, jobs, etc.
- 3 Understand the structure → Check indentation, maps, and lists.
- 4 Read section by section → Focus on one block at a time.
- 5 Look for patterns → Repeated fields usually follow a pattern.
- 6 Check documentation → Use official docs for the specific tool.
- 7 Validate and test → Validate syntax and deploy/test safely.

KEY TAKEAWAY



YAML is the backbone of modern DevOps. Mastering YAML helps you read, write, debug, and automate infrastructure with confidence.

EXAMPLE YAML (KUBERNETES DEPLOYMENT)

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: web-app
5   labels:
6     app: web-app
7 spec:
8   replicas: 3
9   selector:
10    matchLabels:
11      app: web-app
12   template:
13     metadata:
14       labels:
15         app: web-app
16   spec:
17     containers:
18       - name: web
19         image: nginx:1.25
20         ports:
21           - containerPort: 80
```

Annotations:

- 1, 2: API version of the resource, Type of resource
- 3, 4, 5, 6: Metadata about the resource (name, labels, etc.)
- 8, 9, 10, 11: Desired state of the resource (replicas, selector, etc.)
- 12, 13, 14, 15: Pod template (what pods should look like)
- 17, 18, 19, 20, 21: Container configuration (image, ports, etc.)

2. YAML SYNTAX AND STRUCTURE



1. KEY-VALUE PAIRS

YAML is made of keys and values.

EXAMPLE

```
name: veriqlta
role: DevOps Engineer
active: true
```



2. STRINGS

Text values are called strings.

EXAMPLE

```
title: YAML Handbook
author: VERIQTA
message: "Welcome to
YAML for DevOps"
```

123

3. NUMBERS

YAML supports integers and decimals.

EXAMPLE

```
replicas: 3
port: 8080
cpu: 0.5
version: 1.25
```



4. BOOLEANS

Boolean values are true or false.

EXAMPLE

```
enabled: true
debug: false
ssl: true
```



5. NULL VALUES

Null represents empty or no value.

EXAMPLE

```
description: null
owner: ~
optional_field:
```



6. COMMENTS

Use # to add comments. They are ignored by YAML.

EXAMPLE

```
# This is a comment
name: veriqlta # Inline comment
# Another comment
role: DevOps Engineer
```



7. NESTED STRUCTURES

YAML supports nested maps and lists.

EXAMPLE

```
person:
  name: veriqlta
  skills:
    - YAML
    - Kubernetes
    - CI/CD
```



8. INDENTATION RULES

Indentation defines structure in YAML.

- Use consistent spaces
- Indentation shows relationships
- No braces or brackets are required



9. SPACES VERSUS TABS

Always use spaces, not tabs. Tabs can break YAML.

✓ CORRECT (SPACES)

```
server:
  host: localhost
  port: 8080
  enabled: true
```

✗ INCORRECT (TABS)

```
server:
<TAB>host: localhost
<TAB>port: 8080
<TAB>enabled: true
```



10. READING YAML FROM TOP TO BOTTOM

YAML is read from top to bottom. The order matters.

1. Start from the top-level key.
2. Move down line by line.
3. Follow indentation to understand hierarchy.
4. Lists are read in the order they appear.
5. Later values can override earlier ones in some tools.

EXAMPLE

```
app: # 1
  name: veriqlta # 2
  version: 1.0 # 2
  features: # 2
    - ci # 3
    - cd # 3
    - monitoring # 3
```



11. COMMON SYNTAX MISTAKES

✗ MISSING COLON

WRONG

```
name veriqlta
role: DevOps
```

CORRECT

```
name: veriqlta
role: DevOps
```

✗ WRONG INDENTATION

WRONG

```
server:
  host: localhost
  port: 8080
```

CORRECT

```
server:
  host: localhost
  port: 8080
```

✗ MIXING TABS AND SPACES

WRONG

```
server:
<TAB>host: localhost
port: 8080
```

CORRECT

```
server:
  host: localhost
  port: 8080
```

✗ LIST ITEM WITHOUT DASH

WRONG

```
skills:
  YAML
  Kubernetes
```

CORRECT

```
skills:
  - YAML
  - Kubernetes
```

✗ UNQUOTED SPECIAL CHARACTERS

WRONG

```
message: value: 100%
path: C:\\new\\file
```

CORRECT

```
message: "value: 100%"
path: "C:\\new\\file"
```

✗ WRONG DATA TYPE

WRONG

```
replicas: "three"
enabled: "yes"
```

CORRECT

```
replicas: 3
enabled: true
```



3. LISTS, MAPS, AND NESTED DATA



1. YAML SEQUENCES (LISTS)

A sequence is an ordered list of items.

fruits:

- apple
- banana
- orange
- mango



2. YAML MAPPINGS (MAPS)

A mapping is a collection of key-value pairs.

user:

```
name: veriqlta
role: DevOps Engineer
active: true
team: Platform
```



3. LISTS OF OBJECTS

A list where each item is a mapping (object).

servers:

```
- name: web-01
  ip: 10.0.0.10
  role: web
- name: db-01
  ip: 10.0.0.20
  role: database
```



4. NESTED MAPPINGS

A mapping inside another mapping.

database:

```
host: localhost
port: 5432
credentials:
  user: admin
  password: secret
```



5. COMBINING LISTS AND MAPS

The most powerful YAML structures combine lists and maps.

applications:

```
- name: api
  image: api:1.0
  ports:
    - 8080
    - 8443
  env:
    - name: ENV
      value: production
    - name: DEBUG
      value: "false"
```



6. UNDERSTANDING - (DASH)

The dash (-) indicates a list item. Items are part of a sequence.

items:

```
- item1
- item2
- item3
```

→ List (Sequence)
→ First item
→ Second item
→ Third item



REMEMBER:

Each item must align at the same indentation level under the key.



7. UNDERSTANDING : (COLON)

The colon (:) separates a key from its value.

key: : value

↕ ↕ ↕
Key Separator Value



REMEMBER:

A space is required after the colon for valid YAML.



8. CONFIGURATION HIERARCHY

YAML represents hierarchy using indentation. The deeper the indent, the lower the level.

```
root                    → Level 0 (Top)
parent                 → Level 1
  child                → Level 2
    grandchild         → Level 3 (Deepest)
```



REMEMBER:

Indentation defines relationships, not brackets.



KEY TAKEAWAYS

- ✓ Lists (sequences) use -
- ✓ Maps (mappings) use key: value
- ✓ Indentation defines hierarchy
- ✓ Combine lists and maps for real-world configs
- ✓ Dash (-) creates list items
- ✓ Colon (:) separates keys and values
- ✓ Read YAML from top to bottom
- ✓ Practice reading complex nested structures



9. PRACTICAL DEVOPS CONFIGURATION EXAMPLE

A real-world example combining lists, maps, and nested data.

```
app:
  name: payment-service
  version: 2.1.0
  environment: production
  replicas: 3
  image:
    repository: myrepo/payment
    tag: v2.1.0
  ports:
    - 8080
    - 8443
  resources:
    requests:
      cpu: "250m"
      memory: "256Mi"
    limits:
      cpu: "500m"
      memory: "512Mi"
  env:
    - name: LOG_LEVEL
      value: "info"
    - name: FEATURE_X
      value: "enabled"
```

- Application Information
- Container Image Configuration
- Port List
- Resource Configuration
- Environment Variables

4. STRINGS, QUOTES, AND MULTILINE VALUES

1. PLAIN STRINGS

Unquoted text without special characters.

EXAMPLE

```
app: veriqta
env: production
region: us-east-1
owner: DevOps Team
```

2. SINGLE QUOTES

Treats everything literally.
No escape sequences.

EXAMPLE

```
message: 'It's a great day'
path: '/usr/local/bin'
regex: '^d+\. \d+\. \d+$'
```

3. DOUBLE QUOTES

Allows escape sequences
and special characters.

EXAMPLE

```
message: "It's a great day"
path: "C:\Program Files\App"
newline: "Line1\nLine2"
tabbed: "Col1\tCol2"
```

4. WHEN QUOTING MATTERS

Quote values when they may be misinterpreted.

EXAMPLE

```
version: "1.0" # not 1
port: "8080" # keep leading 0
on: "yes" # not boolean
value: "null" # not null
```

5. SPECIAL CHARACTERS

Certain characters can break YAML if not quoted.

SPECIAL CHARACTERS

: , # { } [] & * ! | > ' "

EXAMPLE

```
url: "https://example.com?name=veriqta&env=prod"
query: "key: value"
json: "{\"name\": \"veriqta\"}"
command: "echo \"Hello: World\""
```

6. MULTILINE STRINGS OVERVIEW

Use block scalars for long text.

TYPE	SYNTAX	DESCRIPTION	PRESERVES NEWLINES	FOLDS LINES
Literal Block		Preserves line breaks exactly as written.	✓ YES	✗ NO
Folded Block	>	Folds line breaks into spaces.	✗ NO	✓ YES

7. LITERAL BLOCK |

Keeps line breaks exactly.

EXAMPLE

```
script: |
#!/bin/bash
echo "Starting deployment"
echo "Step 1 complete"
echo "Step 2 complete"
echo "Deployment done"
```

RESULT (PRESERVED)

```
#!/bin/bash
echo "Starting deployment"
echo "Step 1 complete"
echo "Step 2 complete"
echo "Deployment done"
```

8. FOLDED BLOCK >

Converts line breaks to spaces.

EXAMPLE

```
description: >
This is a long description
that spans multiple lines
but will be folded into
a single paragraph.
```

RESULT (FOLDED)

This is a long description
that spans multiple lines
but will be folded into
a single paragraph.

9. ENVIRONMENT-VARIABLE-LIKE VALUES

YAML is ideal for environment variables.

EXAMPLE

```
env:
- name: DATABASE_URL
  value: "postgres://user:pass@db:5432/app"
- name: LOG_LEVEL
  value: "INFO"
- name: FEATURE_X
  value: "true"
```

10. URLS, PORTS, VERSIONS, AND TIMESTAMPS

Quote values to avoid unwanted type casting.

EXAMPLES

```
url: "https://veriqta.dev/api/v1"
port: "8080" # keep as string
version: "2.1.0" # not 2.1 (float)
build: "00123" # keep leading zeros
time: "2025-07-19T10:30:00Z"
date: "2025-07-19"
```

11. YAML PARSING SURPRISES

YAML can interpret values in unexpected ways.

VALUE WRITTEN	YAML INTERPRETS AS	HOW TO FIX
yes	boolean (true)	"yes"
no	boolean (false)	"no"
on	boolean (true)	"on"
off	boolean (false)	"off"
null	null	"null"
~	null	"~"
1.0	float (1)	"1.0"
0123	octal (83)	"0123"

12. QUOTING BEST PRACTICES

- ✓ Use plain strings for simple values.
- ✓ Use single quotes for literal text without escapes.
- ✓ Use double quotes when you need escape sequences.
- ✓ Quote values that look like numbers, booleans, nulls, or timestamps.
- ✓ Quote URLs, regex, JSON, commands, and paths with special characters.
- ✓ Use | for scripts, configs, certificates, and content with line breaks.
- ✓ Use > for long descriptions and text paragraphs.



KEY TAKEAWAY

Correct use of strings, quotes, and multiline values prevents parsing errors and ensures your configuration behaves exactly as intended in production.



5. YAML DATA TYPES AND TYPE PITFALLS

T 1. STRINGS

A sequence of characters represented as text.

EXAMPLE

```
name: veriqta
role: "DevOps Engineer"
path: '/usr/local/bin'
message: Hello World
```

123 2. INTEGERS

Whole numbers without decimals.

EXAMPLE

```
replicas: 3
port: 8080
max_retries: 5
timeout: 60
```

1.23 3. FLOATING-POINT NUMBERS

Numbers with a decimal point.

EXAMPLE

```
cpu: 0.5
threshold: 0.75
ratio: 1.25
percent: 99.9
```

✓ 4. BOOLEANS

Logical values: true or false.

EXAMPLE

```
enabled: true
debug: false
ssl: true
maintenance: false
```

⊗ 5. NULL

Represents an empty or missing value.

EXAMPLE

```
description: null
owner: ~
optional_field:
```

≡ 6. ARRAYS (SEQUENCES)

Ordered lists of values.

EXAMPLE

```
fruits:
  - apple
  - banana
  - orange
  - mango
```

{ } 7. OBJECTS (MAPPINGS)

Key-value pairs grouped together.

EXAMPLE

```
database:
  host: localhost
  port: 5432
  user: admin
  password: secret
```

⚙ 8. AUTOMATIC TYPE INTERPRETATION

YAML automatically converts values to the appropriate data type.

EXAMPLE

```
true      → boolean (true)
123       → integer (123)
1.5       → float (1.5)
null      → null
[a, b]    → array
{k: v}    → object
```

” 9. QUOTING AMBIGUOUS VALUES

Quote values that could be misinterpreted by YAML.

VALUE (UNQUOTED)	INTERPRETED AS	SAFE (QUOTED)	INTERPRETED AS
yes	boolean (true)	"yes"	string
no	boolean (false)	"no"	string
on	boolean (true)	"on"	string
off	boolean (false)	"off"	string
null	null	"null"	string
123	integer (123)	"123"	string
0123	octal (83)	"0123"	string
1.0	float (1.0)	"1.0"	string
~	null	"~"	string

🏷 10. VERSION NUMBERS

Version numbers can be misinterpreted if not quoted.

VALUE (UNQUOTED)	INTERPRETED AS	SAFE (QUOTED)	INTERPRETED AS
1.0	float (1.0)	"1.0"	string
2.1.0	float (2.1)	"2.1.0"	string
v1.2.3	string	"v1.2.3"	string
1e3	scientific (1000)	"1e3"	string
2025-07-19	date	"2025-07-19"	string
2025-07-19T10:30:00Z	timestamp	"2025-07-19T10:30:00Z"	string

012 11. NUMERIC-LOOKING STRINGS

Values that look like numbers but should remain strings.

EXAMPLE

```
account_id: "001234567890"
zip_code: "02139"
build_number: "007"
phone: "+2348012345678"
```

📋 12. SUMMARY OF YAML DATA TYPES

TYPE	EXAMPLE	YAML REPRESENTATION
String	"hello"	hello OR "hello"
Integer	42	42
Float	3.14	3.14
Boolean	true / false	true OR false
Null	null / ~	null OR ~
Array	[a, b, c]	- a - b - c
Object	{k: v}	k: v

⚠ 13. WHY TYPE MISTAKES BREAK AUTOMATION

Incorrect types can cause errors, failed deployments, and unexpected behavior.

- ✗ A string "3" is not the same as integer 3
- ✗ A boolean "false" is not the same as false
- ✗ A null value may cause a missing field error
- ✗ A wrong type can break conditionals in templates
- ✗ APIs and tools expect specific data types
- ✗ Schema validation will fail
- ✗ Automation tools may behave unexpectedly

✓ KEY TAKEAWAYS

- ✓ YAML has seven core data types: string, integer, float, boolean, null, array, object.
- ✓ YAML automatically interprets values unless instructed otherwise.
- ✓ Always quote values that are ambiguous.
- ✓ Version numbers, IDs, and numeric-looking strings should be quoted.
- ✓ Type mistakes are a common source of deployment and automation failures.
- ✓ Understand the type you intend before writing YAML.



TIP

When in doubt, quote it!
It's safer to store a value as a string than to break your configuration.



6. ANCHORS, ALIASES, AND MERGE PATTERNS



1. YAML ANCHORS (&)

anchors allow you to name a block of YAML so it can be reused.

EXAMPLE

```
defaults: &defaults
  image: nginx:1.25
  restart: always
env:
  - LOG_LEVEL=info
  - TZ=UTC
```



2. ALIASES (*)

Aliases reference an anchor to reuse the same block.

EXAMPLE

```
web:
  <<: *defaults
  ports:
    - "80:80"
worker:
  <<: *defaults
  command: run-worker
```



3. MERGE KEY (<<)

The merge key (<<) lets you merge mappings from anchors.

EXAMPLE

```
base: &base
image: python:3.11
restart: unless-stopped
env:
  - APP_ENV=prod
  - TZ=UTC
api:
  <<: *base
  ports:
    - "8080:8080"
```



4. REUSING CONFIGURATION

Reuse common settings across services, jobs, or environments.

EXAMPLE

```
services:
  db: &db
  image: postgres:15
  restart: always
  environment:
    POSTGRES_DB: app
    POSTGRES_USER: app
    POSTGRES_PASSWORD: secret
  cache: *db
  worker: *db
```



5. REDUCING DUPLICATION

anchors and merges keep YAML DRY and easier to maintain.

BEFORE (DUPLICATED)

```
web:
  image: nginx:1.25
  restart: always
  env:
    - LOG_LEVEL=info
    - TZ=UTC
api:
  image: nginx:1.25
  restart: always
  env:
    - LOG_LEVEL=info
    - TZ=UTC
```

AFTER (WITH ANCHORS)

```
defaults: &defaults
  image: nginx:1.25
  restart: always
  env:
    - LOG_LEVEL=info
    - TZ=UTC
web:
  <<: *defaults
api:
  <<: *defaults
```



6. SHARED CONFIGURATION BLOCKS

Store shared settings in one place and reference them across multiple parts of the configuration.

EXAMPLE

```
common-env: &common-env
  - LOG_LEVEL=info
  - TZ=UTC
  - APP_VERSION=1.0.0

services:
  api:
    image: myapp:1.0
    environment: *common-env
  worker:
    image: myapp:1.0
    environment: *common-env
```



7. ENVIRONMENT-SPECIFIC OVERRIDES

Use merge patterns to override only what changes per environment.

EXAMPLE

```
base: &base
image: myapp:1.0
restart: unless-stopped
env:
  - LOG_LEVEL=info
  - TZ=UTC
resources:
  limits:
    cpu: "500m"
    memory: "512Mi"

prod:
  <<: *base
  env:
    - LOG_LEVEL=warn
  resources:
    limits:
      cpu: "1000m"
      memory: "1024Mi"

dev:
  <<: *base
  env:
    - LOG_LEVEL=debug
```



8. READABILITY VERSUS EXCESSIVE REUSE

anchors improve maintainability, but too much reuse can reduce readability.

GOOD PRACTICES

- ✓ Reuse logical, stable blocks
- ✓ Use meaningful anchor names
- ✓ Keep overrides small and clear
- ✓ Prefer clarity over clever reuse

AVOID OVERUSE

- ✓ Too many nested merges
- ✓ Over-abstracting simple configs
- ✓ Hard-to-trace dependencies
- ✓ Using anchors for one-time reuse



9. TOOL COMPATIBILITY CONSIDERATIONS

Most modern DevOps tools support anchors and merges, but some have limitations.

TOOL	ANCHORS & MERGES SUPPORT
Kubernetes	✓ Supported
Docker Compose	✓ Supported
Ansible	✓ Supported
GitHub Actions	✓ Supported
Helm (values.yaml)	✓ Supported
AWS CloudFormation	✗ Not Supported
Terraform (YAML providers)	⚠ Limited / Tool Dependent



TIP: Always test your YAML with the specific tool and version you are using.



10. PRODUCTION EXAMPLE (KUBERNETES DEPLOYMENT)

Real-world example using anchors, aliases, and merge patterns.

```
defaults: &defaults
image: myapp:1.0.0
imagePullPolicy: IfNotPresent
resources:
  requests:
    cpu: "200m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
env: &common-env
  - name: APP_ENV
    value: production
  - name: TZ
    value: UTC
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: web
          <<: *defaults
          env: *common-env
          ports:
            - containerPort: 8080
```



WHAT WE ACHIEVED

- Common settings defined once
- Reused with aliases
- Merged and overridden cleanly
- Easy to scale across environments

KEY TAKEAWAY



anchors (&), aliases (*), and Merge key (<<) help you DRY your YAML, reuse configuration safely, and manage complex DevOps setups across environments with clarity.

VERIQTA



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

7. READING COMPLEX YAML LIKE AN ENGINEER

1. FINDING THE ROOT STRUCTURE

- The root is the top-level key
- Everything else is a child
- Start here to understand the whole document

EXAMPLE (ROOT IS "apiVersion")

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: webapp
5 spec:
```

ROOT
(Top Level)

2. FOLLOWING INDENTATION LEVELS

- Indentation shows hierarchy
- More indent = deeper level
- Use spaces (not tabs)

INDENTATION GUIDE

0 spaces → Level 0 (Root)
 2 spaces → Level 1 (Child)
 4 spaces → Level 2 (Grandchild)
 6 spaces → Level 3 (Great-Grandchild)
 8 spaces → Level 4

✓ ALWAYS USE SPACES. NEVER TABS.

3. IDENTIFYING OBJECTS AND ARRAYS

OBJECT (MAPPING)

Key: Value pairs

```
metadata:
  name: webapp
  labels:
    app: webapp
    env: prod
```

ARRAY (SEQUENCE)

Items in a list

```
containers:
  - name: nginx
    image: nginx:1.25
  - name: sidecar
    image: busybox
```

💡 Mapping = {} style (key: value)
 Sequence = [] style (dash - items)

4. PARENT-CHILD RELATIONSHIPS

- A child belongs to the key above it
- Follow indentation to know the parent

EXAMPLE

```
spec:
  template:
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

← child of spec
 ← child of template
 ← child of spec
 ← child of containers
 ← child of item

5. READING DEEPLY NESTED CONFIGURATION

- Go level by level
- Understand each block before moving deeper

EXAMPLE (NESTED 4 LEVELS DEEP)

```
1 spec:
2   template:
3     spec:
4       containers:
5         - name: web
6           ports:
7             - containerPort: 80
8       resources:
9         limits:
10          cpu: "500m"
```

→ Level 0
 → Level 1
 → Level 2
 → Level 3
 → Level 4

6. RECOGNIZING REPEATED PATTERNS

- Look for repeated blocks
- They indicate lists of similar items

EXAMPLE (PATTERN: CONTAINERS)

```
containers:
  - name: nginx
    image: nginx:1.25
    ports:
      - containerPort: 80
  - name: sidecar
    image: busybox
    command: ["sleep", "3600"]
```

✓ Same structure repeated for each item.
 This is an array of objects.

7. UNDERSTANDING UNFAMILIAR YAML QUICKLY

- 1 Find the root key
- 2 Scan top-level keys
- 3 Identify major sections
- 4 Expand one section at a time
- 5 Look for patterns
- 6 Focus on what matters now
- 7 Ignore what you don't need (for now)

TOP-LEVEL SCAN EXAMPLE

```
apiVersion: apps/v1
kind: Deployment
metadata: {...}
spec:
  replicas: 3
  selector: {...}
  template: {...}
  strategy: {...}
```

8. TURNING YAML INTO A MENTAL TREE

- Visualize YAML as a tree structure

MENTAL TREE (EXAMPLE)

```
spec (root)
├── template
│   └── spec
│       ├── containers
│       │   ├── [0] (name: nginx)
│       │   └── [1] (name: sidecar)
│       └── volumes
│           └── [0] (name: data)
```

💡 TIP
 If you can draw the tree in your head, you truly understand the YAML.

9. PRACTICAL CONFIGURATION WALKTHROUGH

Let's read a complete Kubernetes Deployment YAML.

EXAMPLE YAML

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: webapp
5   labels:
6     app: webapp
7 spec:
8   replicas: 3
9   selector:
10    matchLabels:
11      app: webapp
12   template:
13     metadata:
14       labels:
15         app: webapp
16   spec:
17     containers:
18       - name: nginx
19         image: nginx:1.25
20         ports:
21           - containerPort: 80
```

1 Root Structure
 apiVersion and kind define what this is.
 2 Metadata Section
 Information about the object.
 3 Spec Section
 Desired state of the Deployment.
 4 Template (Pod Blueprint)
 Defines the Pods that will be created.
 5 Containers List
 Array of container objects.
 6 Container Details
 Each container has its own settings.

KEY TAKEAWAYS

- ✓ Start from the root and read top to bottom.
- ✓ Use indentation to understand hierarchy.
- ✓ Identify objects (key: value) and arrays (dash -).
- ✓ Follow parent-child relationships.
- ✓ Look for repeated patterns to understand lists.
- ✓ Build a mental tree to understand complex YAML.
- ✓ Focus on the parts you care about first.
- ✓ Practice makes YAML reading second nature.

COMMON CHECKLIST

- ✓ Identify root
- ✓ Scan top-level keys
- ✓ Expand sections one at a time
- ✓ Check indentation
- ✓ Identify arrays vs objects
- ✓ Find key patterns
- ✓ Build mental tree
- ✓ Understand the purpose

8. YAML VALIDATION AND TROUBLESHOOTING

1. SYNTAX VALIDATION

Checks if YAML follows correct structure and formatting rules.

CHECK

```
# Using Python
python -c "import yaml, sys;
yaml.safe_load(open('file.yaml'))"

# Using yq
yq eval '.' file.yaml
```

✓ Valid syntax = YAML can be parsed.

2. SCHEMA VALIDATION

Checks if YAML matches a defined schema (structure + data types).

COMMON USES

- Kubernetes manifests
- CI/CD pipeline definitions
- CloudFormation, OpenAPI, etc.

TOOLS

- kubeconform, kubeval (K8s)
- JSON Schema validators
- Spectral (OpenAPI)

3. PARSER ERRORS

YAML parser cannot understand the file due to syntax issues.

EXAMPLE

```
yaml.scanner.ScannerError: while scanning
a simple key
  in "file.yaml", line 5, column 1
could not find expected ':'
  in "file.yaml", line 5, column 12
```

Usually caused by missing colon, wrong indentation, or bad characters.

4. COMMON SYNTAX PROBLEMS AND HOW TO FIX THEM

1. INDENTATION FAILURES

Wrong number of spaces breaks structure.

WRONG

```
spec:
  containers:
    - name: nginx
      image: nginx:1.25
```

RIGHT

```
spec:
  containers:
    - name: nginx
      image: nginx:1.25
```

2. DUPLICATE KEYS

Later key overwrites earlier key silently.

WRONG

```
metadata:
  name: app
  name: app-v2 # duplicate
  labels:
    app: web
```

RIGHT

```
metadata:
  name: app
  labels:
    app: web
```

3. MISSING COLONS

A missing colon breaks the key-value pair.

WRONG

```
apiVersion: apps/v1
kind Deployment # missing :
metadata:
  name: webapp
```

RIGHT

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
```

4. INCORRECT LIST STRUCTURE

Mixing list and map incorrectly.

WRONG

```
ports:
  - 80
  port: 8080 # invalid
```

RIGHT

```
ports:
  - containerPort: 80
  - containerPort: 8080
```

5. UNEXPECTED DATA TYPES

Type is different from what is expected.

WRONG

```
replicas: "3" # string
enabled: "false"
resources:
  limits:
    cpu: "500m"
```

RIGHT

```
replicas: 3
enabled: false
resources:
  limits:
    cpu: 500m
```

6. INVISIBLE FORMATTING PROBLEMS

- Tabs instead of spaces
- Trailing spaces
- Non-breaking spaces
- Special Unicode characters (BOM, ZWSP)
- Wrong line endings (CRLF vs LF)
- Mixed indentation

💡 These are hard to see but can break YAML parsing or cause unexpected behavior.

7. USING YAMLLINT

yamllint checks style, formatting, and potential issues.

INSTALL

```
pip install yamllint
```

USAGE

```
yamllint file.yaml
yamllint -d relaxed file.yaml # less strict
```

COMMON CHECKS

- ✓ Indentation
- ✓ Line length
- ✓ Trailing spaces
- ✓ Duplicate keys
- ✓ Empty values
- ✓ Document start/end
- ✓ Key ordering (optional)

8. USING PARSERS AND VALIDATORS

Common tools to parse and validate YAML.

TOOL	PURPOSE	EXAMPLE COMMAND
yq	Parse, query, and edit YAML	yq eval '.' file.yaml
yamllint	Lint and style check	yamllint file.yaml
kubeconform	Validate K8s manifests	kubeconform -f file.yaml
kubeval	Validate K8s manifests	kubeval file.yaml
jq (via yq)	Convert YAML ↔ JSON	yq eval -o=json file.yaml
JSON Schema	Schema validation	ajv validate -s schema.json -d file.yaml

10. QUICK REFERENCE CHECKLIST

- ✓ YAML starts with correct root key.
- ✓ Indentation uses spaces, not tabs.
- ✓ Every key has a colon.
- ✓ Lists use dash (-) with proper indentation.
- ✓ No duplicate keys.
- ✓ Values are of correct data type.
- ✓ No extra spaces or invisible characters.
- ✓ File passes yamllint.
- ✓ Schema validation (if applicable) passes.
- ✓ YAML parses successfully.



TIP

Always validate YAML before deploying. A small error can cause big failures in production.



BEST PRACTICE

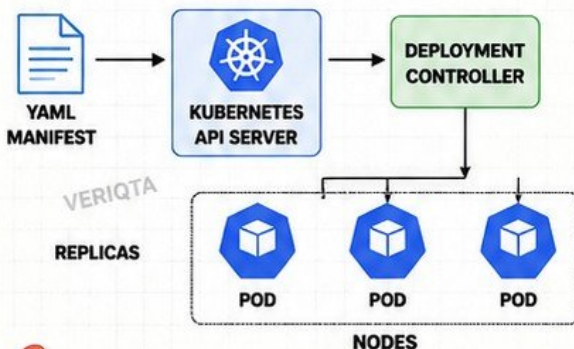
Validate → Lint → Test → Deploy. Never skip steps.

9. TROUBLESHOOTING WORKFLOW

- 1. UNDERSTAND THE ERROR**
Read the full error message carefully. Note the line number and column.
- 2. CHECK SYNTAX**
Run parser (yq / python) to confirm YAML is valid.
- 3. CHECK STRUCTURE**
Validate indentation, colons, lists, and nested levels.
- 4. VERIFY DATA TYPES**
Ensure values are the expected type (string, int, bool, etc.).
- 5. LINT AND VALIDATE**
Run yamllint and schema validator for deeper checks.
- 6. TEST IN REAL ENVIRONMENT**
Apply or test in dry-run mode. Confirm it works as expected.

**apiVersion**API VERSION OF
THE RESOURCE**kind**TYPE OF
KUBERNETES
RESOURCE**metadata**DATA THAT
IDENTIFIES THE
RESOURCE**spec**DESIRED STATE
OF THE
RESOURCE**Understanding
Kubernetes
Manifests****UNDERSTANDING KUBERNETES MANIFESTS**

- KUBERNETES USES DECLARATIVE YAML FILES TO DESCRIBE THE DESIRED STATE.
- KUBERNETES RECONCILES THE ACTUAL STATE TO MATCH THE DESIRED STATE.
- EACH MANIFEST FOLLOWS A STANDARD STRUCTURE.
- MANIFESTS ARE STORED IN GIT AND APPLIED USING KUBECTL OR CI/CD PIPELINES.

HOW A DEPLOYMENT WORKS**KEY TAKEAWAYS**

- ✓ YAML MANIFESTS DEFINE WHAT YOU WANT.
- ✓ KUBERNETES ENSURES THAT STATE IS ACHIEVED AND MAINTAINED.
- ✓ UNDERSTANDING THE STRUCTURE HELPS YOU READ AND DEBUG MANIFESTS FAST.
- ✓ PRACTICE READING AND MODIFYING MANIFESTS DAILY.

COMMON FIELDS EXPLAINED

- **apiVersion**: VERSION OF THE KUBERNETES API.
- **kind**: TYPE OF THE RESOURCE (DEPLOYMENT, SERVICE, ETC.).
- **metadata**: INFORMATION THAT IDENTIFIES THE RESOURCE.
- **spec**: DESIRED STATE OF THE RESOURCE.
- **spec.replicas**: NUMBER OF POD COPIES TO RUN.
- **spec.selector.matchLabels**: HOW KUBERNETES FINDS PODS MANAGED BY THIS DEPLOYMENT.
- **spec.template**: TEMPLATE FOR THE PODS CREATED.
- **ports**: PORTS EXPOSED BY THE CONTAINER.
- **resources**: CPU AND MEMORY REQUESTS AND LIMITS.

DEPLOYMENT YAML EXAMPLE

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: web-app
5    labels:
6      app: web-app
7  spec:
8    replicas: 3
9    selector:
10     matchLabels:
11       app: web-app
12    template:
13     metadata:
14       labels:
15         app: web-app
16     spec:
17       containers:
18         - name: nginx
19           image: nginx:1.25
20           ports:
21             - containerPort: 80
22       resources:
23         requests:
24           cpu: "100m"
25           memory: "128Mi"
26         limits:
27           cpu: "500m"
28           memory: "512Mi"
  
```

API VERSION
OF THE RESOURCETYPE OF
RESOURCERESOURCE
IDENTIFICATIONDESIRED STATE
(REPLICAS, SELECTOR,
TEMPLATE, ETC.)CONTAINER
CONFIGURATION,
PORTS AND
RESOURCES**READING KUBERNETES MANIFESTS CONFIDENTLY**

- ✓ IDENTIFY THE KIND OF RESOURCE.
- ✓ CHECK THE API VERSION.
- ✓ READ METADATA FOR NAME AND LABELS.
- ✓ UNDERSTAND THE DESIRED STATE IN THE SPEC.
- ✓ FOLLOW THE SELECTOR AND LABELS RELATIONSHIP.
- ✓ EXAMINE THE POD TEMPLATE.
- ✓ REVIEW CONTAINER CONFIGURATION.
- ✓ CHECK PORTS, VOLUMES, ENV, AND RESOURCES.
- ✓ VALIDATE SYNTAX BEFORE APPLYING.
- ✓ PREDICT HOW KUBERNETES WILL BEHAVE.
- ✓ CONFIRM THE CREATED RESOURCES AFTER APPLY.

YOUTUBE
veriqtaGITHUB
veriqtaLINKEDIN
veriqtaX
veriqtaINSTAGRAM
veriqtaTELEGRAM
veriqta

ESSENTIAL KUBERNETES RESOURCES CONFIGURED WITH YAML



1. CONFIGMAPS

Store non-sensitive configuration data.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: production
  LOG_LEVEL: info
```



2. SECRETS

Store sensitive data such as passwords and tokens.

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  password: cGFzc3dvcmQ=
  token: dG9kZW4xMjM=
```



3. ENVIRONMENT VARIABLES

Pass configuration to containers at runtime.

```
env:
  - name: APP_MODE
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_MODE
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: app-secret
        key: password
```



4. VOLUMES

Provide storage to containers.

```
volumes:
  - name: data-volume
    emptyDir: {}
  - name: config-volume
    configMap:
      name: app-config
```



5. VOLUME MOUNTS

Mount volumes inside containers.

```
volumeMounts:
  - name: data-volume
    mountPath: /data
  - name: config-volume
    mountPath: /etc/config
    readOnly: true
```



6. SERVICES

Expose a set of Pods as a network service.

```
apiVersion: v1
kind: Service
metadata:
  name: app-service
spec:
  type: ClusterIP
  selector:
    app: web-app
  ports:
    - port: 80
      targetPort: 8080
```



7. INGRESS CONFIGURATION

Manage external HTTP/HTTPS access.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: app-service
                port:
                  number: 80
```



12. COMMON KUBERNETES YAML MISTAKES

- ✗ **INDENTATION ERRORS**
Incorrect indentation can break the YAML.
- ✗ **MISSING COLONS**
Forgetting colons (:) after keys.
- ✗ **WRONG DATA TYPES**
Using string instead of int/boolean.
- ✗ **DUPLICATE KEYS**
YAML does not allow duplicate keys.
- ✗ **INVALID NAMES**
Resource names must follow DNS-1123 rules.
- ✗ **WRONG SELECTORS**
Service selector not matching Pod labels.
- ✗ **EXPOSING SECRETS**
Never store secrets in ConfigMaps or plain text.
- ✗ **RESOURCE OVERCOMMIT**
Setting requests/limits too high or too low.
- ✗ **YAML SYNTAX**
Trailing spaces, tabs or hidden characters.
- 10 **NOT VALIDATING**
Always validate and lint before applying.



8. READINESS PROBE

Determines if container is ready to accept traffic.

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
```



9. LIVENESS PROBE

Detects if container is alive and running.

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 20
```



10. STARTUP PROBE

Gives container time to start before checks.

```
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```



11. RESOURCE REQUESTS AND LIMITS

Ensure proper resource allocation and avoid contention.

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

UNITS GUIDE

CPU

- 1 CPU = 1000m
- 250m = 0.25 CPU

MEMORY

- 1Gi = 1024Mi
- 1Mi = 1024Ki



CPU
500m



MEMORY
512Mi



YOUTUBE
@veriqta



GITHUB
veriqta



LINKEDIN
@veriqta



X
@veriqta



INSTAGRAM
@veriqta



TELEGRAM
@veriqta

1. WORKFLOW STRUCTURE OVERVIEW

	name	Name of the workflow.
	on	Events that trigger the workflow.
	jobs	A set of jobs that run in the workflow.
	runs-on	The runner environment for the job.
	steps	A list of tasks executed in order.
	uses	Use an action from GitHub or Marketplace.
	run	Run shell commands.
	env	Environment variables.
	secrets	Encrypted secrets stored securely.
	expressions	Dynamic values using <code>\${{ }}</code> syntax.
	needs	Define dependencies between jobs.

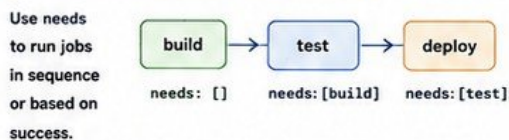
2. GITHUB ACTIONS KEY SECTIONS

name:	Human-readable name for the workflow.
on:	push, pull_request, workflow_dispatch, schedule, etc.
jobs:	Container for all jobs in the workflow.
runs-on:	ubuntu-latest, windows-latest, macos-latest, or self-hosted.
steps:	Tasks executed sequentially in a job.
uses:	Reference an action: owner/repo@version
run:	Execute shell commands.
env:	Set environment variables.
secrets:	Access encrypted secrets.
\${{ }}:	Use expressions for dynamic values.
needs:	Create job dependencies.

3. EXPRESSIONS EXAMPLES

<code>\${{ github.ref }}</code>	Current branch or tag.
<code>\${{ github.sha }}</code>	Commit SHA.
<code>\${{ github.event_name }}</code>	Event that triggered the workflow.
<code>\${{ env.VAR_NAME }}</code>	Access environment variable.
<code>\${{ secrets.SECRET_NAME }}</code>	Access a secret.
<code>\${{ steps.step_id.outputs.output_name }}</code>	Use output from a previous step.
<code>\${{ contains(github.ref, 'main') }}</code>	Expression example.

4. DEPENDENCIES BETWEEN JOBS



5. PRACTICAL CI WORKFLOW EXAMPLE

```
1 name: CI Pipeline
2
3 on:
4   push:
5     branches:
6       - main
7   pull_request:
8     branches:
9       - main
10
11 env:
12   NODE_VERSION: '20'
13   APP_ENV: 'production'
14
15 jobs:
16   build:
17     runs-on: ubuntu-latest
18     outputs:
19       build_status: ${ steps.build_step.outputs.status }
20     steps:
21       - name: Checkout code
22         uses: actions/checkout@v4
23
24       - name: Setup Node.js
25         uses: actions/setup-node@v4
26         with:
27           node-version: ${ env.NODE_VERSION }
28
29       - name: Install dependencies
30         run: npm ci
31
32       - name: Run tests
33         id: build_step
34         run: npm test && echo "status=success" >> $GITHUB_OUTPUT
35
36   test:
37     needs: build
38     runs-on: ubuntu-latest
39     steps:
40       - name: Run lint
41         run: npm run lint
42       - name: Upload test results
43         uses: actions/upload-artifact@v4
44         with:
45           name: test-results
46           path: test-results/
47
48   deploy:
49     needs: [build, test]
50     if: ${ success() }
51     runs-on: ubuntu-latest
52     environment: production
53     steps:
54       - name: Deploy to server
55         run: echo "Deploying application..."
56       - name: Notify success
57         run: echo "Deployment completed successfully!"
```

KEY HIGHLIGHTS

- Triggered on push and pull request to main
- Uses Node.js 20
- Installs deps
- Runs tests
- Sets output for next job

DEPENDENCIES

- test waits for build
- deploy waits for build and test
- Runs only if previous jobs succeed

6. ENVIRONMENT VARIABLES AND SECRETS

ENV VARIABLES

Set globally or per job.

```
env:
  GLOBAL_VAR: 'value'
jobs:
  build:
    env:
      JOB_VAR: 'value'
    steps:
      - run: echo $GLOBAL_VAR $JOB_VAR
```

SECRETS

Store sensitive data securely.

Access with `${{ secrets.NAME }}`

Example usage:

```
- run: echo ${ secrets.API_TOKEN }
- uses: some/action@v1
  with:
    token: ${ secrets.GITHUB_TOKEN }
```

7. COMMON BEST PRACTICES

- ✓ KEEP WORKFLOWS SMALL AND FOCUSED.
- ✓ USE MEANINGFUL JOB AND STEP NAMES.
- ✓ CACHE DEPENDENCIES TO SPEED UP BUILDS.
- ✓ PIN ACTION VERSIONS FOR SECURITY.
- ✓ USE SECRETS FOR SENSITIVE DATA.
- ✓ VALIDATE ENVIRONMENT VARIABLES.
- ✓ USE NEEDS FOR JOB DEPENDENCIES.
- ✓ FAIL FAST AND GET FEEDBACK EARLY.

8. COMMON MISTAKES

- ✗ FORGETTING INDENTATION.
- ✗ USING WRONG EVENT TRIGGERS.
- ✗ NOT PINNING ACTION VERSIONS.
- ✗ HARD-CODING SECRETS.



REMEMBER

A good CI workflow is FAST, RELIABLE, SECURE, and EASY TO MAINTAIN.



YOUTUBE
veriqta



GITHUB
veriqta



LINKEDIN
veriqta



X
@veriqta



INSTAGRAM
@veriqta



TELEGRAM
@veriqta

YAML FOR CI/CD PIPELINES

FROM PIPELINE-AS-CODE TO PRODUCTION DEPLOYMENTS

1. PIPELINE-AS-CODE CONCEPTS



PIPELINE-AS-CODE: DEFINE, VERSION, AND MANAGE YOUR DELIVERY PIPELINE IN YAML.



COLLABORATION: REVIEW CHANGES LIKE APPLICATION CODE.



CONSISTENCY: SAME PIPELINE EVERY TIME.



SCALABILITY: SUPPORTS COMPLEX WORKFLOWS.

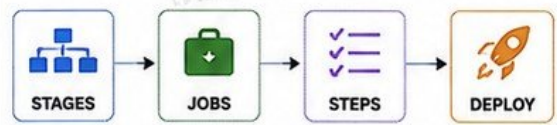


REPRODUCIBILITY: EASY TO RE-RUN AND DEBUG.



AUDITABILITY: FULL HISTORY OF CHANGES.

2. PIPELINE STRUCTURE OVERVIEW



3. CORE BUILDING BLOCKS



STAGES

Logical phases of your pipeline (e.g., Build, Test, Deploy).



JOBS

Units of work that run within a stage.



STEPS

Sequential tasks executed inside a job.



VARIABLES

Dynamic values used across the pipeline.



CONDITIONS

Run tasks or jobs only when conditions are met.



DEPENDENCIES

Define order and dependencies between jobs.



ARTIFACTS

Share files between jobs and stages.



DEPLOYMENT ENVIRONMENTS

Target specific environments for deployments.



REUSABLE CONFIGURATION

Use templates, includes, and shared files.



PIPELINE PATTERNS

Common design patterns for reliable pipelines.



DEBUGGING PIPELINE YAML

Identify and fix issues quickly.

4. SAMPLE CI/CD PIPELINE (GENERIC YAML)

```

1 name: Sample CI/CD Pipeline
2
3 variables:
4   APP_NAME: my-app
5   NODE_VERSION: '20'
6
7 stages:
8   - build
9   - test
10  - package
11  - deploy
12
13 build-job:
14   stage: build
15   runs-on: ubuntu-latest
16   steps:
17     - name: Checkout code
18       uses: actions/checkout@v4
19     - name: Setup Node
20       uses: actions/setup-node@v4
21       with:
22         node-version: ${ env.NODE_VERSION }
23     - name: Install dependencies
24       run: npm ci
25     - name: Build application
26       run: npm run build
27     - name: Upload build artifact
28       uses: actions/upload-artifact@v4
29       with:
30         name: build-output
31         path: dist/
  
```

DEFINE PIPELINE NAME

PIPELINE STAGES IN ORDER

JOB FOR BUILD STAGE

RUNNER ENVIRONMENT

TASKS (STEPS) EXECUTED SEQUENTIALLY

STORE ARTIFACT FOR NEXT STAGES

5. CONDITIONS EXAMPLES

```

# Run only on main branch
if: github.ref == 'refs/heads/main'

# Run only for pull requests
if: github.event_name == 'pull_request'

# Run only when previous job succeeds
if: success()

# Run only when a condition is true
if: ${ env.DEPLOY == 'true' }

# Run when previous job fails
if: failure()
  
```

6. ARTIFACTS EXAMPLE

```

- name: Upload tests report
  uses: actions/upload-artifact@v4
  with:
    name: test-report
    path: reports/
    retention-days: 7

- name: Download build artifact
  uses: actions/download-artifact@v4
  with:
    name: build-output
    path: ./dist
  
```



7. DEPLOYMENT ENVIRONMENTS

```

deploy-job:
  stage: deploy
  runs-on: ubuntu-latest
  environment:
    name: production
    url: https://my-app.com
  steps:
    - name: Deploy to production
      run: ./deploy.sh
  
```



8. REUSABLE CONFIGURATION



USE YAML ANCHORS AND ALIASES.



COMPOSE TEMPLATES FOR REUSE.



CENTRALIZE COMMON STEPS.



REDUCE DUPLICATION.

```

# YAML ANCHOR EXAMPLE
common-steps: &common-steps
- name: Checkout
  uses: actions/checkout@v4
- name: Setup Node
  uses: actions/setup-node@v4

build-job:
  steps: *common-steps
  
```

9. PIPELINE CONFIGURATION PATTERNS

- ✓ LINEAR PIPELINE
- ✓ PARALLEL JOBS
- ✓ FAN-OUT / FAN-IN
- ✓ BUILD → TEST → DEPLOY
- ✓ GATED DEPLOYMENTS
- ✓ BLUE-GREEN DEPLOYMENTS
- ✓ CANARY DEPLOYMENTS

10. DEBUGGING PIPELINE YAML

- ⦿ VALIDATE SYNTAX (yamllint, CI LINT).
- ⦿ CHECK INDENTATION (2 SPACES, NO TABS).
- ⦿ VERIFY STAGE AND JOB NAMES.
- ⦿ REVIEW VARIABLES AND THEIR VALUES.
- ⦿ USE ECHO TO PRINT VARIABLES.
- ⦿ CHECK CONDITIONS LOGIC.
- ⦿ INSPECT ARTIFACTS AND PATHS.
- ⦿ READ LOGS CAREFULLY.
- ⦿ RUN SMALL CHANGES AND TEST.



QUICK CHECKLIST BEFORE COMMITTING

- ✓ SYNTAX IS VALID
- ✓ STAGES AND JOBS ARE IN ORDER
- ✓ ARTIFACTS PATHS ARE VALID
- ✓ DEPENDENCIES ARE DEFINED
- ✓ INDENTATION IS CORRECT
- ✓ VARIABLES ARE SET CORRECTLY
- ✓ CONDITIONS ARE TESTED
- ✓ SECRETS ARE NOT HARDCODED



YOUTUBE
veriqta



GITHUB
veriqta



LINKEDIN
veriqta



X
@veriqta



INSTAGRAM
@veriqta



TELEGRAM
@veriqta

1. COMPOSE FILE STRUCTURE

version: '3.9'

services:
Define application services

volumes:
Define named volumes

networks:
Define custom networks

docker-compose.yml



2. SERVICES

Services are containers that perform a specific role.

```
services:
  web:
    image: nginx:1.25
    ports:
      - "80:80"
```

3. IMAGES

Use official images from registries.

```
image: nginx:1.25
image: postgres:15
```

4. BUILDS

Build custom images from Dockerfile.

```
build:
  context: ./app
  dockerfile: Dockerfile
```

5. PORTS

Map container ports to host ports.

```
ports:
  - "8080:80"
  - "5432:5432"
```

6. ENVIRONMENT VARIABLES

Pass configuration to containers at runtime.

```
environment:
  - APP_ENV=production
  - DB_HOST=db
  - DB_USER=admin
```

7. VOLUMES

Persist and share data between containers and host.

```
volumes:
  - ./data:/var/lib/data
  - db_data:/var/lib/postgresql/data
```

8. NETWORKS

Connect services to communicate securely.

```
networks:
  - app-network
```

9. DEPENDENCIES

Control startup order between services.

```
depends_on:
  - db
  - redis
```

10. HEALTH CHECKS

Ensure containers are running as expected.

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost"]
  interval: 30s
  timeout: 10s
  retries: 3
```

11. MULTIPLE SERVICES

Run multiple containers together as a single application.

```
web (frontend)
api (backend)
db (database)
redis (cache)
nginx (reverse proxy)
```

12. PRACTICAL APPLICATION STACK EXAMPLE

A COMPLETE WEB APPLICATION STACK

```
1 version: '3.9'
2 services:
3   web:
4     build:
5       context: ./frontend
6       dockerfile: Dockerfile
7     ports:
8       - "3000:3000"
9     environment:
10      - REACT_APP_API_URL=http://api:5000
11     depends_on:
12       - api
13     networks:
14       - app-network
15
16   api:
17     build:
18       context: ./backend
19       dockerfile: Dockerfile
20     ports:
21       - "5000:5000"
22     environment:
23       - NODE_ENV=production
24       - DB_HOST=db
25       - DB_USER=admin
26       - DB_PASSWORD=secret
27     depends_on:
28       - db
29       - redis
30     networks:
31       - app-network
32
33   db:
34     image: postgres:15
35     environment:
36       - POSTGRES_USER=admin
37       - POSTGRES_PASSWORD=secret
38       - POSTGRES_DB=appdb
39     volumes:
40       - db_data:/var/lib/postgresql/data
41     networks:
42       - app-network
```

FRONTEND SERVICE

Built from source and exposes port 3000

BACKEND API SERVICE

Built from source and communicates with DB & Redis

DATABASE SERVICE

Uses official image and persists data with a volume

ADDITIONAL SERVICES

```
redis:
  image: redis:7
  ports:
    - "6379:6379"
  networks:
    - app-network

nginx:
  image: nginx:1.25
  ports:
    - "80:80"
  depends_on:
    - web
  volumes:
    - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
  networks:
    - app-network
```

VOLUMES DEFINITION

```
volumes:
  db_data:
```



NETWORKS DEFINITION

```
networks:
  app-network:
    driver: bridge
```



13. COMMON DOCKER COMPOSE YAML MISTAKES

- ✗ INCORRECT INDENTATION (USE SPACES, NOT TABS)
- ✗ FORGETTING VOLUME DEFINITIONS
- ✗ WRONG PORT MAPPINGS
- ✗ NETWORK NAME MISMATCHES
- ✗ MISSING DEPENDENCIES
- ✗ HEALTH CHECK COMMAND ERRORS
- ✗ INCORRECT ENVIRONMENT VARIABLE SYNTAX
- ✗ USING LATEST TAG IN PRODUCTION



TIP

Always validate your compose file using:
docker-compose config
before running it.

1. INVENTORY VERSUS PLAYBOOKS



INVENTORY

Defines the hosts and groups.

- Where Ansible runs.
- Static or dynamic.

hosts.ini (Inventory Example)

```
[web]
web1 ansible_host=10.0.0.11
web2 ansible_host=10.0.0.12

[db]
db1 ansible_host=10.0.0.21

[all:vars]
ansible_user=ubuntu
```



PLAYBOOKS

Defines what to do on the hosts.

- Written in YAML.
- Idempotent and repeatable.

site.yml (Playbook Example)

```
- hosts: web
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
```

2. PLAYBOOK STRUCTURE

```
---
- name: Install and start nginx
  hosts: web
  become: yes
  vars:
    nginx_port: 80
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
        enabled: yes
  handlers:
    - name: Restart nginx
      service:
        name: nginx
        state: restarted
```

PLAY NAME

TARGET HOSTS

PRIVILEGE ESCALATION

VARIABLES

TASKS (ACTIONS)

HANDLERS

3. HOSTS



- Define target machines or groups.
- Use groups for organization.
- Can target multiple groups in one play.

Example

```
- hosts: web,db
  become: yes
  tasks:
    - debug:
        msg: "Hello from
          {{ inventory_hostname }}"
```



4. TASKS

- A list of actions executed in order.
- Each task has a name and a module.

Example

```
- name: Create directory
  file:
    path: /opt/app
    state: directory
    mode: '0755'
```



5. MODULES

- Modules are Ansible's building blocks.
- Each module performs a specific function.

Example

```
- name: Install package
  apt:
    name: nginx
    state: present
    update_cache: yes
```



6. VARIABLES

- Store values and reuse them across tasks.
- Can be defined at many levels.

Example

```
vars:
  app_user: deploy
  app_dir: /opt/app

- name: Create app directory
  file:
    path: "{{ app_dir }}"
    state: directory
```



7. HANDLERS

- Tasks that run only when notified.
- Useful for restarts or reloads.

Example

```
handlers:
- name: Restart nginx
  service:
    name: nginx
    state: restarted

# Notify handler
- name: Update nginx config
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart nginx
```



8. ROLES

- Organize playbooks into reusable components.
- Standard directory structure.

Role Directory Example

```
myrole/
├── tasks/
├── handlers/
├── vars/
├── defaults/
├── templates/
└── files/
```



9. CONDITIONALS

- Run tasks only when conditions are true.
- Use when, unless.

Example

```
- name: Install Apache
  apt:
    name: apache2
    state: present
    when: ansible_os_family == "Debian"

- name: Start Apache
  service:
    name: apache2
    state: started
    when: apache_installed is defined
```



10. LOOPS

- Repeat a task for a list of items.
- Use loop or with_items.

Example

```
- name: Create multiple users
  user:
    name: "{{ item }}"
    state: present
  loop:
    - alice
    - bob
    - charlie

# With items (legacy)
- name: Create groups
  group:
    name: "{{ item }}"
    with_items:
      - devops
      - admins
```



11. PRIVILEGE ESCALATION

- Run tasks with elevated privileges.
- Use become.

Example

```
- hosts: all
  become: yes
  tasks:
    - name: Install package
      apt:
        name: vim
        state: present
    - name: Copy config
      copy:
        src: app.conf
        dest: /etc/app.conf
        owner: root
        group: root
        mode: '0644'
```



12. READING AN ANSIBLE PLAYBOOK

- ✓ READ THE PLAY NAME
Understand the purpose of the play.
- ✓ CHECK THE HOSTS
Identify the target machines or groups.
- ✓ LOOK AT BECOME (PRIVILEGES)
Know if root access is required.
- ✓ REVIEW VARIABLES
Find reusable values used in tasks.
- ✓ FOLLOW THE TASKS IN ORDER
See what actions will be executed.
- ✓ CHECK CONDITIONS AND LOOPS
Know when and how tasks run.
- ✓ LOOK AT HANDLERS
See what will run after changes.
- ✓ UNDERSTAND THE OUTCOME
Visualize the end state of the system.

Quick Reading Example

```
- name: Setup web server
  hosts: web
  become: yes
  vars:
    app_port: 8080
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Copy config
      template:
        src: nginx.j2
        dest: /etc/nginx/nginx
        src.conf
      notify: Restart nginx
  handlers:
    - name: Restart nginx
      service:
        name: nginx
        state: restarted
```

13. KEY TIPS FOR ANSIBLE YAML

- ✓ INDENTATION IS CRITICAL (2 SPACES).
- ✓ ALWAYS USE MODULES, NOT SHELL COMMANDS.
- ✓ KEEP TASKS SMALL AND FOCUSED.
- ✓ USE VARIABLES TO AVOID REPEAT.

- ✓ USE HANDLERS FOR RESTARTS AND RELOADS.
- ✓ TEST PLAYBOOKS IN STAGES.
- ✓ USE ROLES FOR REUSABILITY.
- ✓ VALIDATE YAML SYNTAX BEFORE RUNNING.

- ✓ USE --CHECK TO DRY RUN CHANGES.
- ✓ USE -V OR -VVV FOR DEBUGGING.
- ✓ KEEP PLAYBOOKS IDEMPOTENT.



TIP

Great playbooks are readable, reusable, and easy to debug.



YOUTUBE
veriqta



GITHUB
veriqta



LINKEDIN
veriqta



X
@veriqta



INSTAGRAM
@veriqta



TELEGRAM
@veriqta

1. ENVIRONMENTS OVERVIEW



DEVELOPMENT

- For local development
- Quick changes
- Debug friendly
- Lower resources



TESTING

- For QA and tests
- Feature validation
- Shared testing
- Isolated data



STAGING

- Pre-production
- Mirrors production
- Final validation
- Performance tests



PRODUCTION

- Live environment
- Highly available
- Secure and monitored
- No direct changes

2. BASE CONFIGURATION

Create a common base configuration shared by all environments.

```
# base/config.yaml
app:
  name: my-app
  image: my-app:1.0.0
  replicaCount: 2
service:
  type: ClusterIP
  port: 8080
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "200m"
    memory: "256Mi"
```

3. OVERRIDES PER ENVIRONMENT

Override only what is different per environment.

```
# dev/override.yaml
app:
  replicaCount: 1
  image: my-app:dev
resources:
  requests:
    cpu: "50m"
    memory: "64Mi"
```

```
# test/override.yaml
app:
  replicaCount: 2
  image: my-app:test
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
```

```
# staging/override.yaml
app:
  replicaCount: 3
  image: my-app:staging
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
```

```
# prod/override.yaml
app:
  replicaCount: 5
  image: my-app:prod
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
```

4. AVOID DUPLICATED CONFIGURATION

Use a base file + small overrides instead of full copies.

❌ AVOID (DUPLICATION)

```
# dev/config.yaml (full copy)
...
# test/config.yaml (full copy)
...
# staging/config.yaml (full copy)
...
# prod/config.yaml (full copy)
...
```

✅ PREFERRED (BASE + OVERRIDES)

```
base/config.yaml
dev/override.yaml
test/override.yaml
staging/override.yaml
prod/override.yaml
```

5. ENVIRONMENT VARIABLES

Inject environment-specific values using variables.

Example with placeholders:

```
database:
  host: ${DB_HOST}
  port: ${DB_PORT}
  user: ${DB_USER}
  password: ${DB_PASSWORD}
```

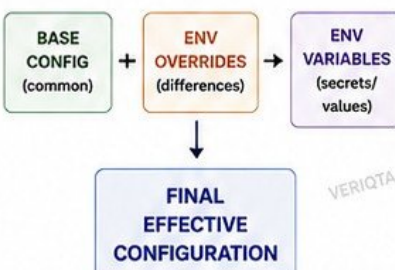
Example .env (per environment):

```
.env.dev
.env.test
.env.staging
.env.prod
```



6. CONFIGURATION LAYERING

Configuration is applied in layers from base to environment.



7. KUBERNETES OVERLAYS (KUSTOMIZE)

Use Kustomize overlays for clean environment management.

```
k8s/
|-- base/
|   |-- deployment.yaml
|   |-- service.yaml
|   |-- customization.yaml
|-- overlays/
|   |-- dev/
|   |   |-- customization.yaml
|   |-- staging/
|   |   |-- customization.yaml
|   |-- prod/
|   |   |-- customization.yaml
```

Each overlay changes only what is needed.



8. HELM VALUES FILES

Helm uses values.yaml + environment specific values files.

```
values.yaml          # base values
values-dev.yaml      # dev overrides
values-staging.yaml  # staging overrides
values-prod.yaml     # prod overrides
```

```
helm install my-app ./chart -f values.yaml -f values-prod.yaml
```

9. MANAGING CONFIGURATION DRIFT

Drift happens when live configuration differs from version controlled YAML.

CAUSES

- Manual changes
- Out-of-band edits
- Missing updates
- Inconsistent processes



DRIFT
DETECTED

PREVENTION

- ✓ GitOps
- ✓ Immutable infra
- ✓ Automated deploys
- ✓ Continuous diff checks
- ✓ Policy enforcement

10. BEST PRACTICES

- ✓ KEEP A SMALL, CLEAN BASE CONFIGURATION.
- ✓ OVERRIDE ONLY WHAT CHANGES.
- ✓ USE ENVIRONMENT VARIABLES FOR SENSITIVE DATA.
- ✓ NEVER HARDCODE SECRETS.
- ✓ VALIDATE YAML BEFORE DEPLOYMENT.
- ✓ USE KUSTOMIZE OR HELM FOR COMPLEX SETUPS.
- ✓ AUTOMATE DEPLOYMENTS (CI/CD OR GITOPS).
- ✓ MONITOR AND DETECT CONFIG DRIFT.
- ✓ DOCUMENT ENVIRONMENT-SPECIFIC VALUES.

11. EXAMPLE: FINAL MERGED CONFIG (PRODUCTION)

```
app:
  name: my-app
  image: my-app:prod
  replicaCount: 5
service:
  type: ClusterIP
  port: 8080
database:
  host: prod-db.example.com
  port: 5432
  user: appuser
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
  limits:
    cpu: "1000m"
    memory: "1Gi"
```

✅ This is the final effective configuration used in production.



TIP

Keep configurations DRY, layered, and version controlled.
Small overrides, big impact!

16. SECRETS AND SECURITY IN YAML

PROTECT SENSITIVE INFORMATION. SECURE YOUR CONFIGURATIONS.



WHY SECRETS END UP IN YAML

- Quick testing and demos
- Copying from dashboards or consoles
- Convenience during development
- Lack of understanding of secure practices
- Legacy configurations
- Poor secret management processes



HARDCODED CREDENTIAL RISKS

- Credentials exposed in repositories
- Anyone with access can steal secrets
- Secrets leaked in pull requests and logs
- Increased attack surface
- Compliance and audit failures
- Potential data breaches and downtime

COMMON TYPES OF SECRETS IN YAML



API KEYS

Keys used to access third-party APIs and services.



TOKENS

Authentication tokens for services and platforms.



PASSWORDS

Database, user, and system passwords.



KUBERNETES SECRETS

Secrets stored as Kubernetes Secret objects.



CI/CD SECRETS

Credentials used in pipelines and build systems.



SECURE WAYS TO HANDLE SECRETS

- ✓ USE SECRET REFERENCES INSTEAD OF HARDCODING
- ✓ STORE SECRETS IN KUBERNETES SECRETS
- ✓ USE CI/CD SECRET STORES (GITHUB SECRETS, GITLAB CI/CD VARIABLES, ETC.)
- ✓ USE ENVIRONMENT VARIABLES
- ✓ USE SECRET MANAGERS (AWS SECRETS MANAGER, HASHICORP VAULT, AZURE KEY VAULT, GOOGLE SECRET MANAGER)
- ✓ ENCRYPT SECRETS AT REST AND IN TRANSIT
- ✓ ROTATE SECRETS REGULARLY
- ✓ LIMIT ACCESS TO ONLY WHAT IS NECESSARY



SECRET REFERENCES IN YAML

KUBERNETES EXAMPLE:

```
env:  
- name: DB_PASSWORD  
  valueFrom:  
    secretKeyRef:  
      name: db-secret  
      key: password
```

USE REFERENCES TO FETCH SECRETS SECURELY INSTEAD OF HARDCODING THEM.



GIT REPOSITORY EXPOSURE RISKS

- Secrets remain in Git history even after deletion
- Forks and copies can expose secrets
- Public repositories are fully visible
- CI/CD logs may print secrets accidentally
- Developers may share configs insecurely



ONCE A SECRET IS COMMITTED, IT IS CONSIDERED COMPROMISED. ROTATE IT IMMEDIATELY.



PREVENTING ACCIDENTAL SECRET COMMITS

- NEVER commit secrets to Git
- Add sensitive files to .gitignore
- Use pre-commit hooks and secret scanning tools
- Enable secret scanning in GitHub/GitLab
- Review pull requests carefully
- Rotate any secret that may have been exposed
- Educate your team on secret hygiene



SECURITY REVIEW CHECKLIST

- | | |
|--|---|
| ☐ ARE ANY SECRETS HARDCODED IN YAML? | ☐ ARE SECRETS ENCRYPTED AT REST AND IN TRANSIT? |
| ☐ ARE KUBERNETES SECRETS USED CORRECTLY? | ☐ IS ACCESS TO SECRETS LIMITED TO REQUIRED SERVICES? |
| ☐ ARE CI/CD SECRETS STORED IN SECURE STORES? | ☐ IS SECRET ROTATION ENABLED? |
| ☐ ARE ENVIRONMENT VARIABLES USED APPROPRIATELY? | ☐ IS THE REPOSITORY PROTECTED FROM EXPOSURE? |
| ☐ ARE SECRET REFERENCES USED INSTEAD OF VALUES? | ☐ HAVE YOU RUN SECRET SCANNING ON THE REPOSITORY? |

17. YAML IN GIT AND GITOPS WORKFLOWS

MANAGE INFRASTRUCTURE AND CONFIGURATION WITH GIT AS THE SOURCE OF TRUTH.



YAML AS VERSION-CONTROLLED INFRASTRUCTURE

- YAML files describe your entire infrastructure and applications.
- Stored in Git repositories for visibility, history, and auditability.
- Enables consistency, traceability, and collaboration.
- Git becomes the single source of truth.



PULL REQUESTS

- All changes are proposed via Pull Requests.
- Review changes before they are merged.
- Automated checks and validations run on every PR.
- Prevents direct changes to main branch.



CODE REVIEW

- Peer review ensures quality and security.
- Validate YAML syntax, structure, and intent.
- Check for best practices and compliance.
- Document decisions in the PR conversation.



CONFIGURATION REPOSITORIES

- Store YAML manifests, Helm charts, Kustomize overlays, and environment configs.
- Organized by environment (dev, staging, prod).
- Example structure:

```
config-repo/
├── clusters/
│   ├── dev/
│   ├── staging/
│   └── prod/
├── apps/
│   ├── my-app/
│   └── another-app/
└── README.md
```



GITOPS PRINCIPLES

- Declarative: Define the desired state in YAML.
- Versioned: Store config in Git.
- Automated: Tools reconcile actual state with desired state.
- Auditable: Every change is traceable.
- Secure: Git access control and policies.

GITOPS WORKFLOW WITH ARGO CD



1. DEVELOPER
Makes changes to
YAML configuration
locally.



2. GIT REPOSITORY
Push changes
and create a
Pull Request.



3. CODE REVIEW
Review, test, and
approve the
changes.



4. MERGE
Merge PR to
main branch.



5. ARGO CD
Detects changes in Git
and syncs to the
cluster automatically.



6. CLUSTER
Desired state
achieved in the
Kubernetes cluster.



ARGO CD AND YAML

- Argo CD watches your Git repository.
- When YAML changes are merged, Argo CD syncs them to the cluster.
- Provides visibility, status, and health.
- Supports automated sync and manual approval.
- Strong integration with Kubernetes.



CONFIGURATION CHANGES THROUGH GIT

- All infrastructure and application changes happen through Git.
- No manual changes on servers or clusters.
- Immutable infrastructure and declarative state.
- Easy collaboration across teams.



ROLLBACKS

- Revert to any previous commit in Git.
- Argo CD syncs the previous state.
- Fast and safe recovery from bad changes.



DRIFT DETECTION

- Argo CD continuously monitors the cluster.
- Detects manual changes or configuration drift.
- Alerts and auto-heal to restore desired state.



PRODUCTION CHANGE CONTROL

- Enforce strict branch protection rules.
- Require reviews and approvals.
- Use policies and automated checks.



AUDIT TRAIL AND HISTORY

- Every change is logged in Git.
- Who changed what, when, and why.
- Complete history for audits and compliance.



BEST PRACTICES

- One change per PR.
- Small, focused commits.
- Validate before merge.
- Monitor sync status and alerts.

18. PRODUCTION YAML DESIGN PRACTICES

CLEAN. CONSISTENT. SECURE. SCALABLE CONFIGURATION.



1. KEEP CONFIGURATION READABLE

- Simplicity beats cleverness.
- Use clear structure and logical grouping.
- Make it easy for anyone to understand at a glance.

```
app:
  name: web-app
  image: nginx:1.25
  ports:
    - "80:80"
```



2. AVOID UNNECESSARY NESTING

- Deep nesting makes YAML harder to read and maintain.
- Flatten the structure where possible.

GOOD ✓

```
resources:
  limits:
    cpu: "500m"
    memory: "512Mi"
```

AVOID ✗

```
spec:
  template:
    spec:
      containers:
        - name: app
          resources:
            limits:
              cpu: "500m"
              memory: "512Mi"
```



3. NAMING CONVENTIONS

- Use clear, descriptive names.
- Be consistent across environments.
- Use lowercase and hyphens.
- Avoid generic names like app, test, or data.

GOOD	AVOID
user-service	app
payment-api	service1
prod-web	myconfig
redis-cache	data



4. COMMENTS THAT ADD VALUE

- Explain the WHY, not the WHAT.
- Document non-obvious decisions.
- Keep comments up to date.

```
# Increase replica count to handle Black Friday traffic
# Based on load testing results (Nov 2024)
replicas: 5
```



5. CONSISTENT FORMATTING

- Use consistent indentation (2 spaces recommended).
- Consistent key ordering helps readability.
- End files with a newline.
- Use tools to auto-format YAML.

```
indent: 2 spaces ✓
lists start with - ✓
align related keys ✓
consistent spacing ✓
```



6. REUSABLE CONFIGURATION

- DRY: Don't Repeat Yourself.
- Extract common values into files or charts.
- Use templates, includes, or values files.

base-config.yaml

```
image: nginx
ports:
  - "80:80"
env:
  - LOG_LEVEL=info
```

used by

service-a.yaml

```
<<: *base
replicas: 2
```

service-b.yaml

```
<<: *base
replicas: 3
```



7. AVOID EXCESSIVE ANCHORS

- Anchors and aliases can improve reuse but overuse reduces readability.
- Use anchors only when they simplify the file.
- Prefer templates or includes for large-scale reuse.

```
# Use sparingly
common: &common
restart: unless-stopped
env:
  - TZ=UTC
service:
  <<: *common
  image: nginx
```



8. SEPARATE CONFIGURATION FROM SECRETS

- Never store secrets in YAML files.
- Reference secrets from secure stores.
- Use environment variables or secret managers.

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: db-secret
        key: password
```



9. VALIDATE BEFORE DEPLOYMENT

- Validate syntax and schema.
- Use linters in CI/CD pipelines.
- Catch issues early, before they reach production.

```
yamllint . ✓
kubeval file.yaml ✓
kubeconform file.yaml ✓
helm lint chart/ ✓
kubectl apply --dry-run=client ✓
```



10. SCHEMA-AWARE EDITING

- Use YAML editors with schema support (Kubernetes, CI/CD, etc).
- Get auto-completion, validation, and error detection.
- Reduces mistakes and speeds up development.



VS Code



IntelliJ IDEA



JetBrains
Yaml Plugin



Lens



11. REVIEWABILITY

- Write YAML that is easy to review.
- Small, focused changes per PR.
- Logical file organization.
- Avoid massive, complex YAML files.

GOOD PR

- One change
- Clear intent
- Easy to review
- Safe to approve



12. MAINTAINABILITY AT SCALE

- Structure repositories for clarity.
- Use modular design.
- Document standards.
- Continuously refactor.
- Build for teams, not just today.

THINK LONG TERM

- Team growth
- Multiple environments
- Multiple services
- Different regions



PRODUCTION YAML DESIGN CHECKLIST

- | | | | |
|--|---|---|---|
| ☒ Readable and Simple | ☒ Helpful Comments | ☒ No Excessive Anchors | ☒ Schema-Aware Editing |
| ☒ No Unnecessary Nesting | ☒ Consistent Formatting | ☒ Secrets Are Externalized | ☒ Easy to Review |
| ☒ Clear Naming Conventions | ☒ Reusable and DRY | ☒ Validated Before Deployment | ☒ Maintainable at Scale |

19. REAL-WORLD YAML TROUBLESHOOTING LAB

DIAGNOSE. FIX. LEARN. APPLY IN PRODUCTION.



HOW TO USE
THIS PAGE



SYMPTOM



BROKEN
YAML



INVESTIGATION



ROOT
CAUSE



CORRECTED
YAML



LESSON

1



KUBERNETES
DEPLOYMENT
FAILS BECAUSE OF
INCORRECT
INDENTATION.

SYMPTOM

- Pod does not start.
- kubectl output shows: "error parsing YAML"
- or "did not find expected key"

BROKEN YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app # wrong
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web
          image: nginx:1.25
```

INVESTIGATION

- Run: `kubectl apply -f deployment.yaml`
- Check error message
- Open file and review indentation
- YAML parser fails near `matchLabels`

ROOT CAUSE

Incorrect indentation under `matchLabels`. `app` should be indented 2 more spaces.

CORRECTED YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web
          image: nginx:1.25
```

LESSON

Indentation is structure in YAML. Use linting and editors with YAML validation.

2



A SERVICE
SELECTOR DOES
NOT MATCH
DEPLOYMENT
LABELS.

SYMPTOM

- Service has no endpoints.
- Traffic does not reach pods.
- kubectl get endpoints shows `<none>`

BROKEN YAML

```
# Service YAML
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: myapp # does not match
  ports:
    - port: 80
      targetPort: 8080
```

INVESTIGATION

- `kubectl get svc web-svc`
- `kubectl get endpoints web-svc`
- Check Deployment labels
- Compare selector vs pod labels

ROOT CAUSE

Service selector (`app: myapp`) does not match Deployment label (`app: web-app`).

CORRECTED YAML

```
# Service YAML
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: web-app # matches
  ports:
    - port: 80
      targetPort: 8080
```

LESSON

Labels and selectors must match exactly. Always verify endpoints.

3



GITHUB ACTIONS
WORKFLOW
DOES NOT
TRIGGER.

SYMPTOM

- Push made to repo but workflow does not run.
- No workflow appears in Actions tab.

BROKEN YAML

```
on: push:
  branches:
    - main # wrong indentation
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hello"
```

INVESTIGATION

- Check Actions tab
- Verify workflow file is in `.github/workflows`
- Validate YAML syntax
- Check indentation under 'push:'

ROOT CAUSE

branches should be indented under push (2 spaces). YAML structure is wrong.

CORRECTED YAML

```
on:
  push:
    branches:
      - main
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hello"
```

LESSON

YAML structure matters. Even small indentation mistakes can stop pipelines from triggering.

4



DOCKER COMPOSE
ENVIRONMENT
VARIABLE IS
INTERPRETED
INCORRECTLY.

SYMPTOM

- Application receives wrong value for environment variable.
- Boolean/number values become unexpected.

BROKEN YAML

```
services:
  app:
    image: myapp:latest
    environment:
      DEBUG: false # interpreted as boolean false
      PORT: 08080 # interpreted as number (8080)
      MESSAGE: # interpreted as 08
```

INVESTIGATION

- Run: `docker compose config`
- Check resolved values
- Print env vars inside container
- Compare types

ROOT CAUSE

Unquoted values are interpreted as boolean or numbers by YAML.

CORRECTED YAML

```
services:
  app:
    image: myapp:latest
    environment:
      DEBUG: "false"
      PORT: "08080"
```

LESSON

Quote values that look like booleans, numbers, or special words.

5



ANSIBLE PLAYBOOK
FAILS BECAUSE
A LIST WAS
WRITTEN AS
A MAPPING.

SYMPTOM

- Playbook fails to run.
- Error about invalid data type.

BROKEN YAML

```
tasks:
  - name: Install packages
    apt:
      name: nginx # should be a list
      state: present
```

INVESTIGATION

- Run: `ansible-playbook playbook.yml`
- Read error message
- Check module documentation
- Validate YAML types

ROOT CAUSE

Ansible expects a list for 'name'. Mapping given instead.

CORRECTED YAML

```
tasks:
  - name: Install packages
    apt:
      name:
        - nginx
        - curl
      state: present
```

LESSON

Understand data types expected by modules. List ≠ Mapping.

6



A PRODUCTION
DEPLOYMENT
CONTAINS AN
ACCIDENTALLY
COMMITTED
SECRET.

SYMPTOM

- Secret or credential found in Git repository.
- Potential security risk.

BROKEN YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_PASSWORD: "MySecret123!"
  API_KEY: "abcd1234efgh5678"
```

INVESTIGATION

- Scan repository for secrets
- Manual code review
- Check commit history
- Use secret scanning tools

ROOT CAUSE

Secrets were committed directly in YAML files.

CORRECTED YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_PASSWORD: "${DB_PASSWORD}"
  API_KEY: "${API_KEY}"
# Store actual secrets in Secret
# manager or Kubernetes Secret.
```

LESSON

Never commit secrets. Use Secret Managers and scanning tools.

TROUBLESHOOTING WORKFLOW CHECKLIST



Reproduce the issue

Check YAML syntax

Validate structure and indentation

Verify values and types

Check references and dependencies

Apply fix and retest

Document lessons learned

VERIQTA

YouTube @veriqta

GitHub @veriqta

LinkedIn @veriqta

X @veriqta

Instagram @veriqta

Telegram @veriqta

20. PRODUCTION YAML CHALLENGE AND ENGINEER CHECKLIST



PRODUCTION YAML CHALLENGE

BUILD AND REVIEW A SMALL PRODUCTION CONFIGURATION SET CONTAINING:



KUBERNETES DEPLOYMENT

Define a Deployment for your application with containers, resources, probes, and environment variables.



KUBERNETES SERVICE

Expose your application using a Service with the correct selector and ports.



CONFIGMAP

Store non-sensitive configuration values in a ConfigMap and mount or reference it in your app.



SECRET REFERENCE

Reference sensitive values from a Kubernetes Secret in your application configuration.



GITHUB ACTIONS CI/CD WORKFLOW

Create a workflow that tests, builds, and deploys your application.



DOCKER COMPOSE

Define a multi-service application with volumes, networks, and environment variables.



ANSIBLE AUTOMATION EXAMPLE

Write an Ansible playbook to perform a deployment or infrastructure task.



FINAL VERIFICATION

- | | |
|---|--|
| ☐ VALIDATE SYNTAX | ☐ INSPECT REFERENCES AND DEPENDENCIES |
| ☐ CHECK INDENTATION | ☐ RUN LINTING |
| ☐ VERIFY DATA TYPES | ☐ REVIEW GIT CHANGES |
| ☐ REVIEW ENVIRONMENT CONFIGURATION | ☐ TEST BEFORE PRODUCTION |
| ☐ CHECK SECRET HANDLING | ☐ PERFORM FINAL CONFIGURATION REVIEW |



★ ONLY AFTER ALL CHECKS ARE COMPLETE AND VERIFIED SHOULD THE CONFIGURATION BE MERGED AND DEPLOYED.



FINAL OUTCOME

BY THE END OF THE HANDBOOK, THE READER SHOULD BE ABLE TO:

- | | |
|---------------------------------------|--|
| ✓ READ COMPLEX YAML CONFIDENTLY | ✓ BUILD CI/CD CONFIGURATION |
| ✓ WRITE CLEAN YAML FROM SCRATCH | ✓ UNDERSTAND DOCKER COMPOSE YAML |
| ✓ UNDERSTAND NESTED CONFIGURATION | ✓ READ ANSIBLE PLAYBOOKS |
| ✓ IDENTIFY COMMON YAML FAILURES | ✓ HANDLE ENVIRONMENT-SPECIFIC CONFIGURATION |
| ✓ VALIDATE AND LINT YAML | ✓ PROTECT SECRETS |
| ✓ TROUBLESHOOT CONFIGURATION PROBLEMS | ✓ REVIEW YAML DURING PULL REQUESTS |
| ✓ WORK WITH KUBERNETES YAML | ✓ USE YAML SAFELY IN PRODUCTION DEVOPS WORKFLOWS |



YAML IS POWERFUL, BUT SMALL MISTAKES CAN CAUSE BIG OUTAGES.
WRITE CLEAN. VALIDATE OFTEN. REVIEW CAREFULLY. DEPLOY CONFIDENTLY.